Food Ordering Management System

PostgreSQL + Python (Tkinter) Edition

A Project Report
in partial fulfilment for the award of the degree of
Master of Computer Application

Submitted by:
Sahil Gupta
UID: 25MCI10266

Submitted to:
Mr. Shalabh Bhatiya
Assistant Professor

25MAM-1A

Chandigarh University
Mohali, Punjab

Acknowledgement

We deem it a pleasure to acknowledge our sense of gratitude to our project guide, Ms Atul Sharma (Assistant Professor, UIC) under whom we have carried out the project work. Her incisive and objective guidance and timely advice encouraged us with constant flow of energy to continue the work.

We wish to reciprocate in full measure the kindness shown by Dr. Krishan Tuli (H.O.D, University Institute of Computing) who inspired us with his valuable suggestions in successfully completing the project work.

We shall remain grateful to Dr. Manisha Malhotra, Additional Director, University Institute of Computing, for providing us a strong academic atmosphere by enforcing strict discipline to do the project work with utmost concentration and dedication.

Finally, we must say that no height is ever achieved without some sacrifices made at some end and it is here where we owe our special debt to our parents and our friends for showing their generous love and care throughout the entire period of time.

Date: 02-11-2025

Place: Chandigarh University, Mohali, Punjab.

By: Sahil Gupta, UID- 25MCI10266.

Abstract

The Food Ordering Management System is a desktop-based application developed in Python using Tkinter for the graphical user interface (GUI) and PostgreSQL as the backend database. The system is designed to automate and simplify the process of food ordering in restaurants, cafés, and small food outlets by providing a digital platform for customers and administrators to manage menu items, customer details, and order transactions efficiently.

This application allows users to browse through categorized food and drink menus, add selected items to a virtual cart, and place orders seamlessly through an intuitive interface. Each order is linked to a specific customer, whose details are stored and managed within the system database. The PostgreSQL backend manages multiple tables such as Customer, Food, Drink, Orders, Order Items, and Payment, ensuring data consistency, reliability, and efficient retrieval of records. The design emphasizes modularity with distinct layers for data management, business logic, and user interaction.

The system improves traditional manual ordering methods by reducing human errors, speeding up order processing, and maintaining a digital record of every transaction. It features transaction-safe checkout, live quantity tracking, a search bar, and a real-time admin panel. Furthermore, it provides administrators with the ability to add or update menu items and view order summaries in real time.

Overall, the Food Ordering App demonstrates the practical integration of Python's GUI capabilities with a full relational database management system, highlighting concepts of modular programming, event-driven architecture, normalization, and user-centered design. It serves as an efficient, scalable, and modern solution to streamline the food ordering process in small to medium-scale food service businesses.

Table of Contents

S.No.	Content	Page No.
1	Introduction	5-6
2	Objectives	6-7
3	Tools Used and Requirements	7-8
4	System Overview	8-12
5	Entity Relationship Diagram	12
6	System Design	13-15
7	PostgreSQL Schema	15-17
8	Code	17-22
9	Result	22-24
10	Output	24-26
11	Conclusion and Future Scope	27-28
12	Bibliography	30

1. Introduction

In today's world, technological advancements have influenced nearly every aspect of human life, and the food industry is no exception. From online food delivery apps to automated restaurant management systems, digital transformation has redefined how customers interact with restaurants and how businesses operate. One of the most common issues faced by restaurants and cafés is the manual handling of food orders, which often leads to inefficiency, delays, and data inconsistency. The Food Ordering Management System has been developed to address these challenges by providing a simple, digital, and automated solution.

This project is a desktop-based food ordering application developed using the Python programming language with Tkinter for the graphical user interface (GUI) and PostgreSQL as the backend database. The system is migrated from a single-file SQLite architecture to a modular PostgreSQL-based design, split into app.py (GUI), db.py (database helpers), and schema.sql (PostgreSQL schema). This allows users to view food and drink menus, select items, add them to a virtual cart, and place an order — all through an interactive and visually appealing interface.

The system bridges the gap between the customer and the restaurant by offering an organized and systematic approach to order management. Traditionally, restaurant staff record customer orders manually on paper or via basic billing systems. This not only increases the chance of human error but also leads to confusion during peak hours, miscommunication between kitchen and front-end staff, and delays in service. The Food Ordering Management System eliminates these issues by automating the entire process — from order placement to record storage — while ensuring accuracy, speed, and simplicity.

From a technical standpoint, the project demonstrates practical implementation of several computer science concepts, including Database Management Systems (DBMS), Event-Driven Programming, Transaction Management, and Graphical User Interface (GUI) Design. PostgreSQL, being a powerful open-source RDBMS, ensures that the system runs efficiently with support for serial keys, cascading relations, index-based retrieval, and transaction-safe operations.

This project is designed keeping in mind the needs of small and medium-scale restaurants, cafés, and canteens. By using open-source technologies (Python + PostgreSQL), the system offers a cost-effective and customizable solution. It can be easily extended to integrate advanced features such as online ordering, real-time updates, and digital payments in the future.

In addition to solving real-world business problems, the project also serves as a learning platform for understanding how GUI-based applications interact with relational databases. It helps in gaining hands-on experience in handling CRUD operations, structuring Python programs, and designing modular software systems following good programming practices.

2. Objectives

The Food Ordering Management System has been developed with the primary goal of creating an efficient, user-friendly, and automated platform for managing food and beverage orders in a restaurant or café environment. The system aims to simplify the traditional manual ordering process, reduce errors, and improve customer service through a digital interface.

2.1 Primary Objectives

To automate the food ordering process: Replace traditional pen-and-paper or verbal ordering systems with a fully digital and automated platform that minimizes manual work and human error.

To develop a user-friendly graphical interface: Design a clean, intuitive, and interactive GUI using Tkinter, allowing both staff and customers to place and manage orders easily.

To efficiently store and manage order data: Implement a PostgreSQL database to handle menu items, customer details, and order records securely with full transactional integrity.

To improve operational efficiency in restaurants: Streamline the ordering workflow — from taking an order to billing and record-keeping — reducing order delays and increasing staff productivity.

To provide accurate billing and order summaries: Automatically calculate total amounts, generate order receipts, and maintain consistency across orders without manual calculation errors.

To create a modular and maintainable system: Build the system with clear modular design (app.py, db.py, schema.sql), separating the interface, logic, and database so that future enhancements can be easily implemented.

2.2 Specific Technical Objectives

- Design and implement a structured PostgreSQL schema with serial primary keys, foreign key constraints, and cascading relations.
- Develop interactive Tkinter GUI components including tabs, treeviews, search bars, and dialog boxes.
- Handle order management functions: add items to cart, remove items, and confirm orders while dynamically updating total costs.
- Integrate backend logic (db.py) with the GUI (app.py) through well-defined Python functions and parameterized SQL queries.
- Ensure data persistence and reliability through PostgreSQL's ACID-compliant transactions.
- Implement live quantity tracking: decrement food/drink stock atomically on checkout using row-level locking (FOR UPDATE).
- Support scalability and future extensions through normalized schema design and modular code.

3. Tools Used and Requirements

Tool / Technology	Purpose / Description
Python 3.10+	Core programming language for GUI development, business logic, and database integration.
Tkinter & ttk	Built-in Python GUI toolkit for designing interactive windows, tabs, treeviews, buttons, frames, and dialogs.
PostgreSQL 13+	Powerful open-source relational DBMS replacing SQLite. Supports SERIAL keys, transactions, cascading FKs, indexes, and row-level locking.
psycopg2-binary	Python adapter for PostgreSQL. Provides connection management, parameterized queries, RealDictCursor, and context-managed transactions.
pandas 2.2+	Optional: used for data analysis, admin analytics, and generating order/sales summaries.
DB Browser / pgAdmin	Graphical tools for viewing and managing PostgreSQL database tables, verifying records, and debugging SQL queries during development.
VS Code / PyCharm / IDLE	Development environments for writing, editing, and testing Python scripts efficiently.
schema.sql	Dedicated SQL file containing the full PostgreSQL DDL schema: table definitions, constraints, and indexes.

3.1 PostgreSQL vs SQLite — Key Differences

Feature	SQLite (Previous)	PostgreSQL (Current)
Architecture	Serverless, single file	Client-server RDBMS
Transactions	Basic	Full ACID with row locking
Primary Keys	INTEGER AUTOINCREMENT	SERIAL (auto-sequence)
Foreign Keys	Optional enforcement	Enforced with CASCADE
Data Types	Loosely typed	Strict typing: NUMERIC, DATE, TIMESTAMP
Indexes	Basic	Named indexes with CREATE INDEX
Concurrency	Single writer	Multi-user with MVCC

4. System Overview

4.1 Overview of the System

The Food Ordering Management System is a desktop-based application developed using Python (Tkinter) for the front end and PostgreSQL for the back end. The project has been migrated from a monolithic SQLite design to a modular, production-grade architecture split across three files: app.py (Tkinter GUI), db.py (PostgreSQL CRUD helpers), and schema.sql (database schema). The main goal of this system is to simplify and automate the process of ordering food and drinks, managing customers, and maintaining transaction records efficiently.

The system allows a customer to:

- View available food and drink items with live quantity tracking,
- Search menu items using the built-in search bar,
- Select and add them to the cart,
- Provide customer details (new or existing),
- Choose a payment method and delivery option,
- And place the order through a transaction-safe checkout flow.

The system automatically calculates the total bill and stores all information in the PostgreSQL database for record-keeping and analysis.

4.2 Database Tables and Their Roles

1. Customer Table

Table Name: customer

Attributes: custid (SERIAL PK), name, phone, address

Functionality: Stores all customer details. Each customer has a unique custid generated automatically by PostgreSQL. The simplified schema stores name, phone, and address, providing a clean link to orders and payments.

2. Cuisine Table

Table Name: cuisine

Attributes: cuisineid (SERIAL PK), cuisinename (UNIQUE)

Functionality: Categorizes the different types of cuisines offered (Italian, Chinese, Mexican, Indian, Japanese). Each cuisine type is uniquely identified by cuisineid, helping organize food items and enable better menu classification.

3. Employee Table

Table Name: employee

Attributes: empid (SERIAL PK), fname, lname, dob, emailid (UNIQUE), pwd, address, phoneno, gender, salary

Functionality: Holds information about all restaurant employees. The empid serves as a Primary Key and connects employees to the Chef and Delivery tables for staff assignment and tracking.

4. Chef Table

Table Name: chef

Attributes: chefid (SERIAL PK), chefname, address, street, phoneno, cuisineid (FK → cuisine), empid (FK → employee), emailid (UNIQUE), pwd, salary

Functionality: Stores details about chefs responsible for food preparation. Each chef is linked to a specific cuisine (cuisineid) and their employee record (empid) with ON DELETE SET NULL cascading, ensuring no orphan records.

5. Ingredient Table

Table Name: ingredient

Attributes: ingid (SERIAL PK), ingname (UNIQUE)

Functionality: Contains the list of ingredients used in food preparation. The UNIQUE constraint prevents duplicates. This table is linked to the food table for recipe tracking and inventory management.

6. Food Table

Table Name: food

Attributes: foodid (SERIAL PK), foodname, price (NUMERIC, CHECK ≥ 0), quantity (INTEGER, CHECK ≥ 0), foodavail, cuisineid (FK), ingid (FK), chefid (FK)

Functionality: Stores all menu food items with price constraints and availability status. In the PostgreSQL version, quantity is atomically decremented on checkout using a FOR UPDATE row-level lock to prevent overselling.

7. Drink Table

Table Name: drink

Attributes: drinkid (SERIAL PK), drinkname, price (NUMERIC, CHECK ≥ 0), quantity (INTEGER, CHECK ≥ 0), drinkavail

Functionality: Stores all drink menu items. Similar structure to the food table with CHECK constraints on price and quantity. Stock is decremented atomically on checkout using the same locking mechanism.

8. Delivery Table

Table Name: delivery

Attributes: delid (SERIAL PK), delname, vehicleno, delcharge (NUMERIC, CHECK ≥ 0), deldate (DATE), deltime (TIME), custid (FK → customer), empid (FK → employee)

Functionality: Stores information about order deliveries including delivery agent, vehicle, charge, and scheduled time. Linked to both customer and employee with ON DELETE SET NULL.

9. Orders Table

Table Name: orders

Attributes: ordid (SERIAL PK), custid (FK → customer, NOT NULL), delid (FK → delivery), totalcost (NUMERIC, CHECK ≥ 0), paymethod, status (DEFAULT 'Placed'), created_at (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)

Functionality: Central order management table. Each order stores the customer, delivery option, total cost, payment method, and status. The created_at timestamp enables chronological order history. Unlike the previous SQLite design, this table uses a separate order_items table for line items (normalized design).

10. Order Items Table (New in PostgreSQL Version)

Table Name: order_items

Attributes: itemid (SERIAL PK), ordid (FK → orders, CASCADE), item_type (CHECK: 'food' or 'drink'), foodid (FK), drinkid (FK), item_name, unit_price, qty, subtotal, with a constraint ensuring item_type consistency.

Functionality: A normalized replacement for the previous monolithic orders table. Each row in order_items stores one line item for an order. ON DELETE CASCADE ensures that if an order is deleted, all its items are also removed automatically. A CHECK constraint enforces that food items have foodid set (and drinkid NULL) and vice versa.

11. Payment Table

Table Name: payment

Attributes: payid (SERIAL PK), custid (FK → customer, NOT NULL), ordid (FK → orders, CASCADE), paymethod, amount (NUMERIC, CHECK ≥ 0), paid_at (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)

Functionality: Records all financial transactions. Each payment is linked to both the order and customer. The paid_at timestamp provides an audit trail. Payment is inserted atomically within the same transaction as order placement to ensure financial consistency.

4.3 PostgreSQL Indexes

The schema defines four indexes for optimized query performance:

Index Name	Table / Column	Purpose
idx_food_name	food(foodname)	Fast menu search by name
idx_drink_name	drink(drinkname)	Fast drink search by name
idx_order_customer	orders(custid)	Quick customer order lookup
idx_order_created	orders(created_at)	Chronological order queries

5. Entity Relationship Diagram

An ER diagram is a visual representation that illustrates the entities (tables) within a database and the relationships between them. In the Food Ordering System, the ER diagram shows how entities like customers, orders, chefs, and ingredients are connected, providing a holistic view of the database structure.

The key relationships in the PostgreSQL schema are:

- customer ↔ orders (1:N) — one customer can have many orders (custid FK, ON DELETE RESTRICT)
- orders ↔ order_items (1:N) — one order contains many line items (ordid FK, ON DELETE CASCADE)
- food ↔ order_items (1:N) — a food item can appear in many order_items (foodid FK)
- drink ↔ order_items (1:N) — a drink item can appear in many order_items (drinkid FK)
- cuisine ↔ food (1:N) — one cuisine type classifies many food items (cuisineid FK)
- chef ↔ food (1:N) — one chef prepares many food items (chefid FK)
- employee ↔ chef (1:1) — each chef is linked to an employee record (empid FK)
- customer ↔ payment (1:N) — one customer can have many payments (custid FK, ON DELETE RESTRICT)
- orders ↔ payment (1:1) — each order has exactly one payment record (ordid FK, ON DELETE CASCADE)
- delivery ↔ orders (1:N) — one delivery slot can serve multiple orders (delid FK, ON DELETE SET NULL)

6. System Design

The Food Ordering Management System is designed to simplify the process of browsing food and drink items, adding them to the cart, placing orders, and maintaining customer and order records. The design follows a modular and layered architecture, ensuring efficiency, scalability, and ease of use. The system integrates a graphical user interface (GUI) built using Python's Tkinter library with a PostgreSQL database for reliable data storage and management.

6.1 Architecture Overview

The system architecture is divided into three main layers:

1. Data Layer (schema.sql + PostgreSQL)

- Manages all interactions with the food_ordering_db PostgreSQL database.
- Stores persistent data and ensures all updates, deletions, and insertions are performed with ACID guarantees.
- Maintains tables: customer, cuisine, employee, chef, ingredient, food, drink, delivery, orders, order_items, payment.
- Enforces data integrity through foreign key constraints, CHECK constraints, UNIQUE constraints, and named indexes.
- Uses SERIAL columns for all primary keys (auto-incremented sequences).

2. Processing Layer (db.py)

- Acts as the core logic unit bridging the GUI and the database.
- Provides a db_config() function reading PostgreSQL credentials from environment variables (PGHOST, PGPORT, PGDATABASE, PGUSER, PGPASSWORD).
- Uses a context-managed get_conn() generator for automatic commit/rollback on success/failure.
- Implements RealDictCursor for dictionary-based row retrieval.
- Key functions:
  - init_db() — runs schema.sql and seeds empty tables with sample data.
  - get_menu(search) — returns food and drink lists with optional ILIKE search.
  - place_order() — transaction-safe checkout: decrements stock with FOR UPDATE lock, inserts order, order_items, and payment in one atomic transaction.
  - get_table_rows(table_name) — whitelist-based admin data viewer.
  - insert_customer(), get_customers(), get_orders(), get_order_items() — CRUD helpers.

3. Presentation Layer (app.py — GUI)

- Provides an intuitive and visually appealing interface for users to interact with the system.
- Built using Tkinter and ttk widgets: Notebook tabs, Treeview tables, Entry fields, Buttons, and Dialogs.
- Tab-based navigation: Menu (Food + Drinks), Cart, and Admin panels.
- Search bar for live menu filtering using ILIKE queries via db.py.
- Cart tab shows items, quantities, subtotals, and total bill updated dynamically.

- Order placement flow: customer ID entry → new customer dialog if needed → delivery selection → payment method → transaction-safe checkout.
- Admin tabs for viewing all database tables: customers, food, drinks, orders, order_items, payments, deliveries.

6.2 Project File Structure

File	Description
app.py	Tkinter GUI — all windows, tabs, treeviews, dialogs, and event handlers.
db.py	PostgreSQL connection manager, seeding logic, and all CRUD/query helper functions.
schema.sql	Complete PostgreSQL DDL: CREATE TABLE statements, CHECK constraints, foreign keys, and indexes.
requirements.txt	Python dependencies: psycopg2-binary≥2.9.9, pandas≥2.2.0.
README.md	Setup instructions, environment variable configuration, and project submission highlights.

7. PostgreSQL Schema

The schema is defined in schema.sql and is automatically executed on first run via init_db(). All tables use SERIAL primary keys, strict CHECK constraints, and proper foreign key cascading.

7.1 Core Table Definitions

```
-- Customer
CREATE TABLE IF NOT EXISTS customer (
    custid SERIAL PRIMARY KEY,
    name VARCHAR(120) NOT NULL,
    phone VARCHAR(20),
    address TEXT
);
```

```
-- Food
CREATE TABLE IF NOT EXISTS food (
    foodid SERIAL PRIMARY KEY,
    foodname VARCHAR(120) NOT NULL,
    price NUMERIC(10,2) NOT NULL CHECK (price >= 0),
    quantity INTEGER NOT NULL DEFAULT 0 CHECK (quantity >= 0),
    foodavail VARCHAR(20) NOT NULL DEFAULT 'Available',
    cuisineid INTEGER REFERENCES cuisine(cuisineid) ON DELETE SET NULL,
    ingid INTEGER REFERENCES ingredient(ingid) ON DELETE SET NULL,
    chefid INTEGER REFERENCES chef(chefid) ON DELETE SET NULL
);
```

```
-- Orders
CREATE TABLE IF NOT EXISTS orders (
    ordid SERIAL PRIMARY KEY,
    custid INTEGER NOT NULL REFERENCES customer(custid) ON DELETE RESTRICT,
    delid INTEGER REFERENCES delivery(delid) ON DELETE SET NULL,
    totalcost NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (totalcost >= 0),
    paymethod VARCHAR(40) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'Placed',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Order Items (normalized line items)
CREATE TABLE IF NOT EXISTS order_items (
    itemid SERIAL PRIMARY KEY,
    ordid INTEGER NOT NULL REFERENCES orders(ordid) ON DELETE CASCADE,
    item_type VARCHAR(10) NOT NULL CHECK (item_type IN ('food', 'drink')),
    foodid INTEGER REFERENCES food(foodid) ON DELETE RESTRICT,
    drinkid INTEGER REFERENCES drink(drinkid) ON DELETE RESTRICT,
    item_name VARCHAR(120) NOT NULL,
```

```
    unit_price NUMERIC(10,2) NOT NULL CHECK (unit_price >= 0),
    qty INTEGER NOT NULL CHECK (qty > 0),
    subtotal NUMERIC(12,2) NOT NULL CHECK (subtotal >= 0)
);
```

7.2 Indexes

```
CREATE INDEX IF NOT EXISTS idx_food_name ON food (foodname);
CREATE INDEX IF NOT EXISTS idx_drink_name ON drink (drinkname);
CREATE INDEX IF NOT EXISTS idx_order_customer ON orders (custid);
CREATE INDEX IF NOT EXISTS idx_order_created ON orders (created_at);
```

8. Code

The project is split into three main source files following the separation of concerns principle:

8.1 db.py — Database Connection and CRUD Helpers

db.py handles all PostgreSQL interactions. It reads credentials from environment variables, provides a context-managed connection, runs the schema on startup, seeds sample data if tables are empty, and exposes clean CRUD functions used by the GUI.

```
# Key imports
import psycopg2
from psycopg2.extras import RealDictCursor
```

```
def db_config() -> dict:
    return {
        'host': os.getenv('PGHOST', 'localhost'),
        'port': os.getenv('PGPORT', '5432'),
        'dbname': os.getenv('PGDATABASE', 'food_ordering_db'),
        'user': os.getenv('PGUSER', 'postgres'),
        'password': os.getenv('PGPASSWORD', 'postgres'),
    }
```

```
@contextmanager
def get_conn():
    conn = psycopg2.connect(**db_config())
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()
```

8.2 Transaction-Safe Checkout (place_order)

The place_order() function is the most critical function. It runs entirely within a single PostgreSQL transaction to ensure atomicity: stock is decremented, the order is inserted, line items are saved, and payment is recorded — all or nothing.

```
def place_order(customer_id, delivery_id, paymethod, cart) -> int:
    with get_conn() as conn:
        with conn.cursor() as cur:
            total = sum(Decimal(str(i['price'])) * i['qty'] for i in cart)
            cur.execute(
                'INSERT INTO orders (custid, delid, totalcost, paymethod, status)'
```

```
                ' VALUES (%s,%s,%s,%s,%s) RETURNING ordid',
                (customer_id, delivery_id, total, paymethod, 'Placed'))
            ordid = cur.fetchone()[0]
            for item in cart:
                # Lock row to prevent overselling
                cur.execute('SELECT quantity FROM food WHERE foodid=%s FOR UPDATE',
                            (item['id'],))
                cur.execute('UPDATE food SET quantity = quantity - %s WHERE
foodid=%s',
                            (item['qty'], item['id']))
                # Insert line item into order_items
                cur.execute('INSERT INTO order_items (...) VALUES (%s,...)', ...)
            # Insert payment record in same transaction
            cur.execute('INSERT INTO payment (custid, ordid, paymethod, amount)
                         VALUES (%s,%s,%s,%s)', (customer_id, ordid, paymethod,
total))
            return ordid
```

8.3 Menu Search with ILIKE

```
def get_menu(search: str = ''):
    if search:
        like = f'%{search.lower()}%'
        foods = fetch_all(
            'SELECT foodid, foodname, price, quantity, foodavail FROM food
             WHERE LOWER(foodname) LIKE %s ORDER BY foodname', (like,))
    else:
        foods = fetch_all('SELECT ... FROM food ORDER BY foodname')
    return foods, drinks
```

9. Result

The Food Ordering Management System was successfully developed and executed using Python (Tkinter GUI) and PostgreSQL as the backend database. The system performed all the intended operations efficiently — including menu display, cart management, customer handling, and transaction-safe order placement — through a clean and user-friendly interface.

1. Application Launch and Interface

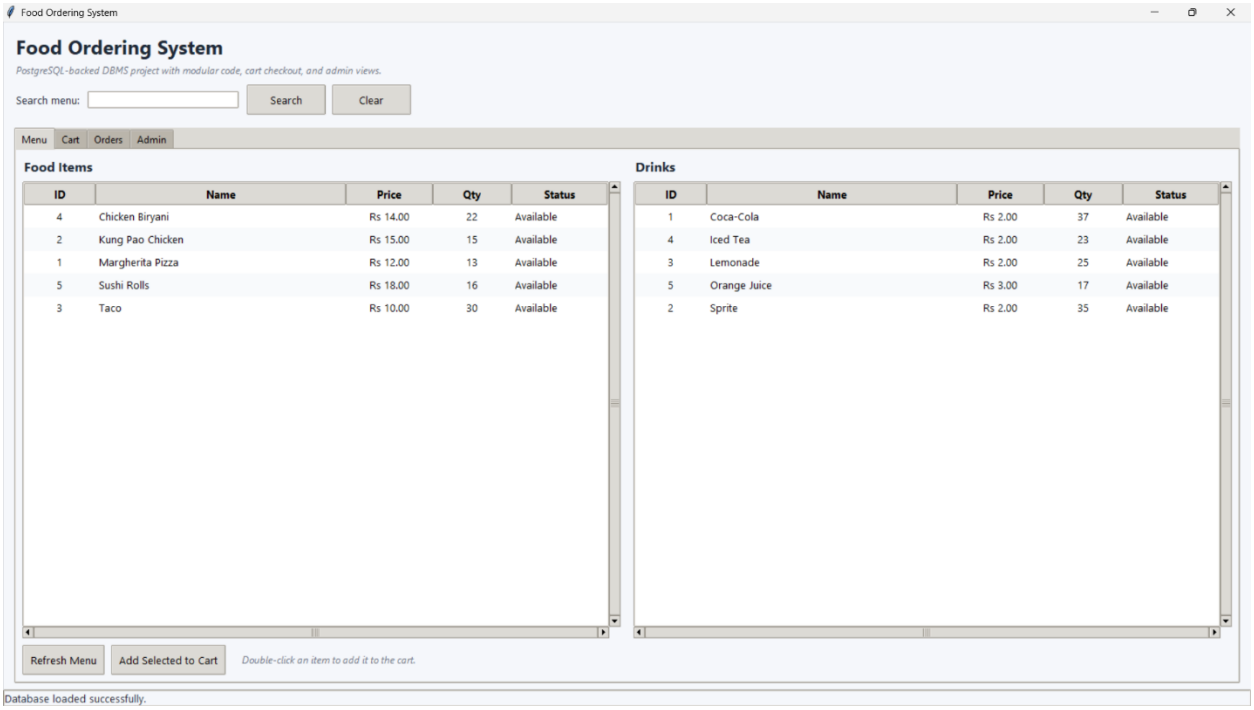
- Upon execution, the application runs init_db() which creates all PostgreSQL tables (if not already present) and seeds sample data.
- The main window opens with tabs: Menu Tab, Cart Tab, and Admin Tab.
- The interface follows a light theme with organized treeviews, buttons, and a search bar.

Home screen

2. Menu Functionality

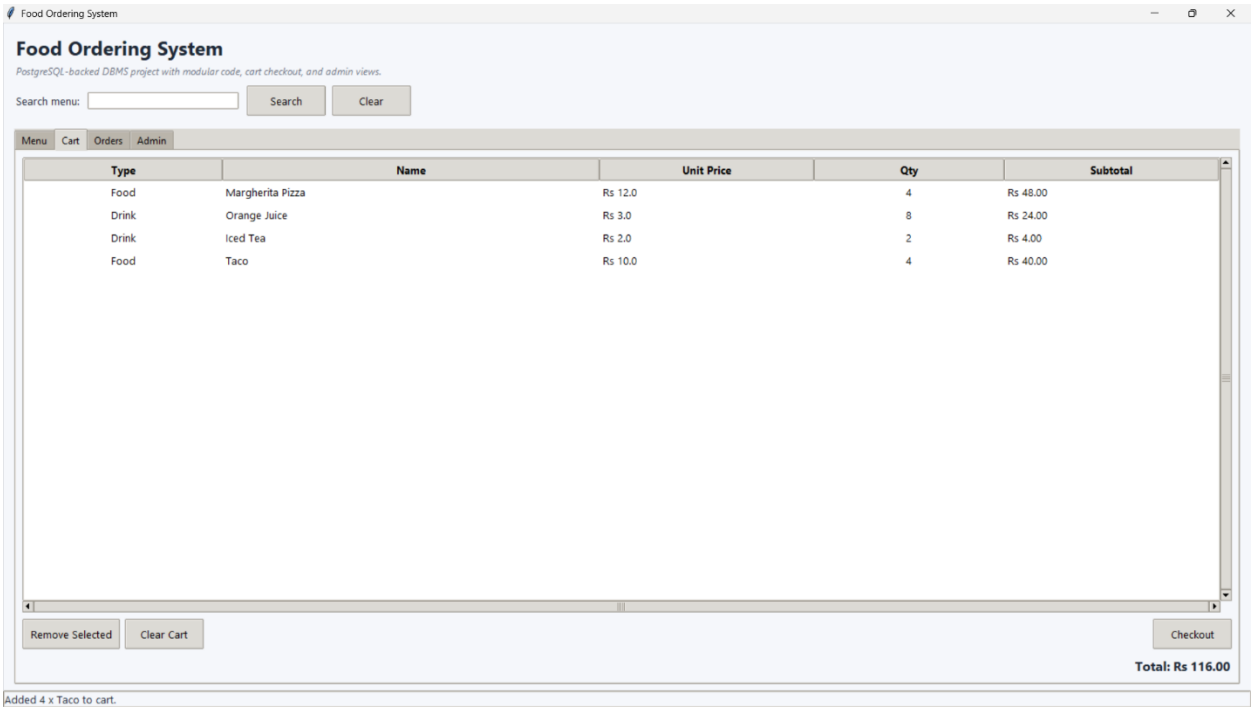
- The Menu tab displays all food and drink items fetched from the PostgreSQL database.
- Each item shows: Item Name, Price, Quantity, Availability.
- Users can use the search bar to filter items using PostgreSQL's ILIKE pattern matching.
- Double-clicking adds items to the cart with quantity dialog.

CART menu bar



3. Cart Functionality

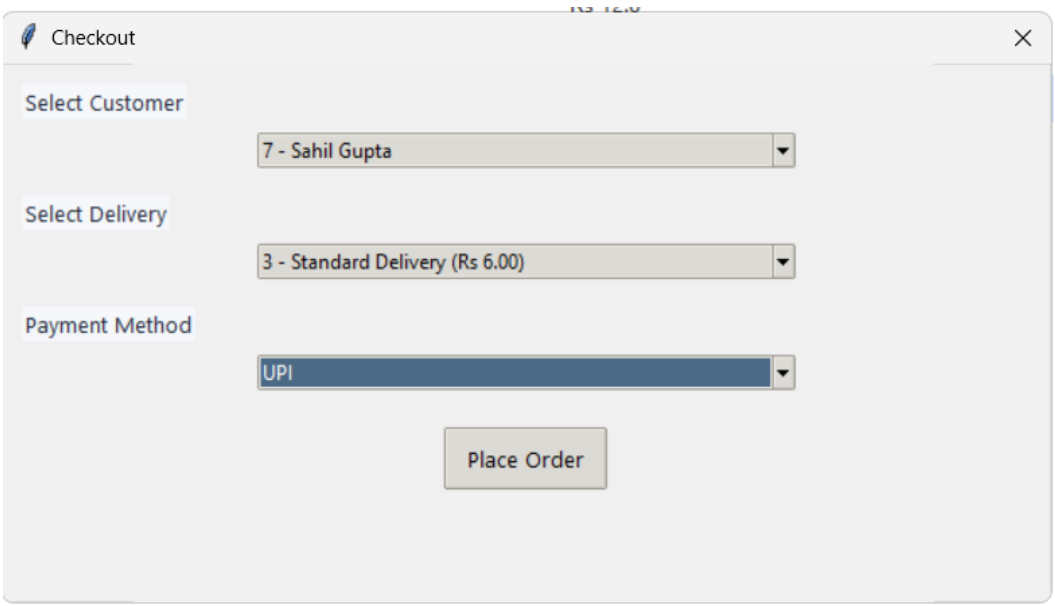
- The Cart tab accurately reflects items added by the user.
- Dynamically updates the subtotal and total bill.
- Users can remove individual items, clear the entire cart, or proceed to checkout.

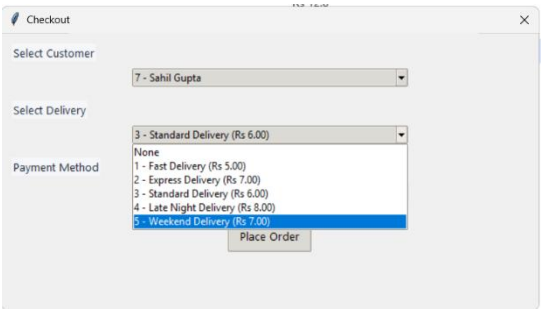


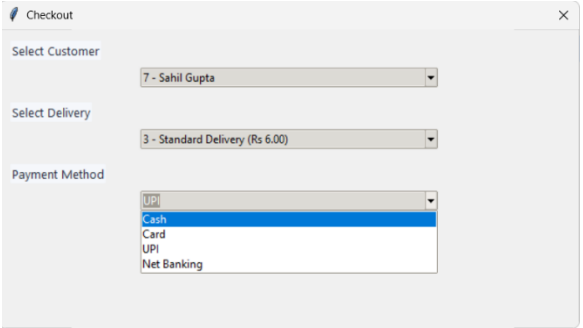
4. Order Placement (Transaction-Safe)

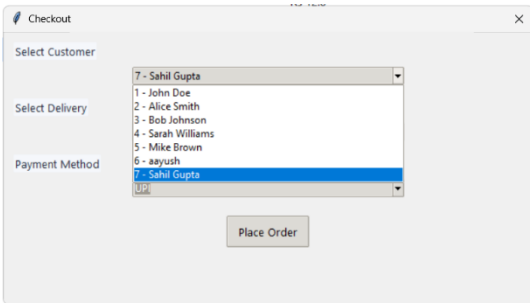
- The system asks for a Customer ID.
- If the customer does not exist, a new customer record is created via insert_customer().
- A delivery option and payment method are selected.
- The entire checkout (stock decrement + order insert + order_items insert + payment insert) executes as a single atomic PostgreSQL transaction.
- On success, a confirmation message is displayed with the Order ID and total bill.

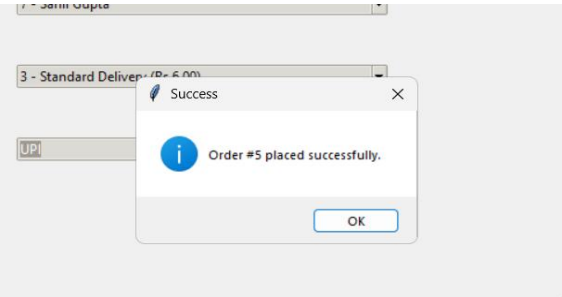
- On failure (e.g., insufficient stock), the transaction is rolled back and the user is notified.









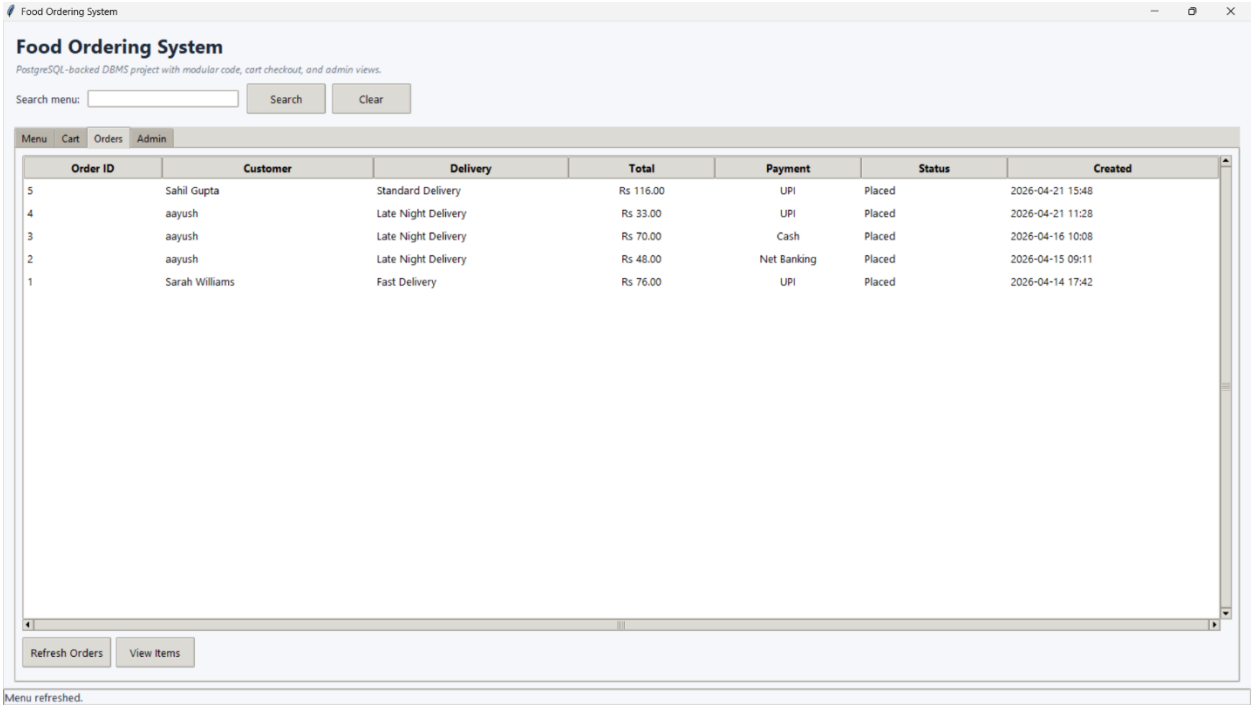


5. Database Operations

The PostgreSQL database (food_ordering_db) contains all normalized tables. All CRUD operations function correctly:

- customer — customer details.
- food / drink — menu items with live quantity updates.

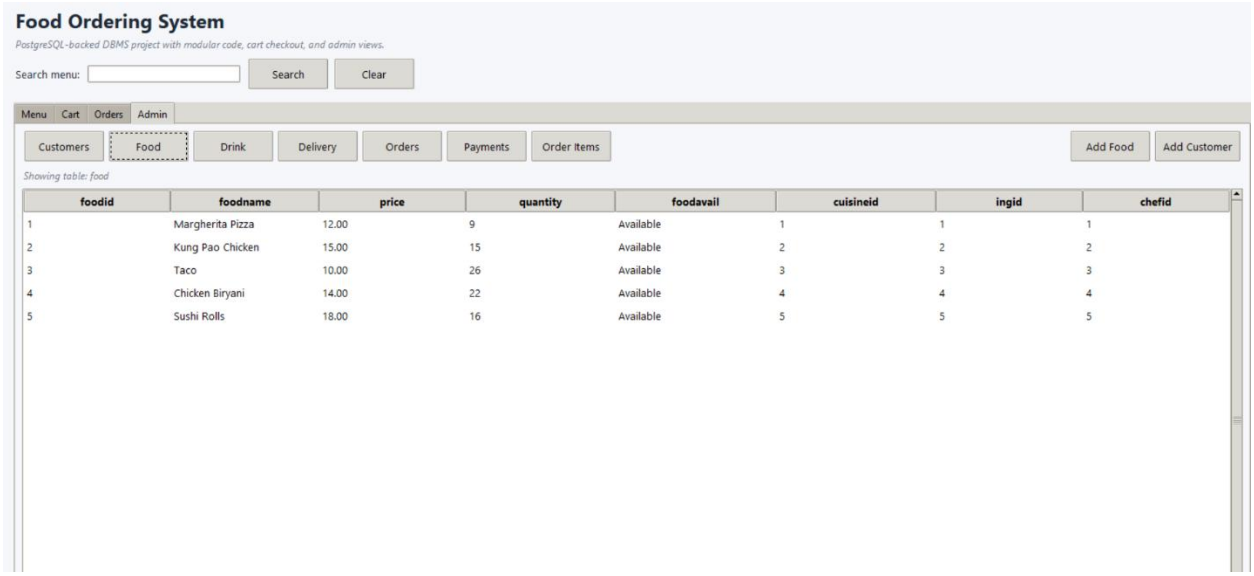
- orders — order summary with status and timestamp.
- order_items — normalized line items per order.
- payment — transaction records with paid_at timestamps.



Order ID	Customer	Delivery	Total	Payment	Status	Created
5	Sahil Gupta	Standard Delivery	Rs 116.00	UPI	Placed	2026-04-21 15:48
4	aayush	Late Night Delivery	Rs 33.00	UPI	Placed	2026-04-21 11:28
3	aayush	Late Night Delivery	Rs 70.00	Cash	Placed	2026-04-16 10:08
2	aayush	Late Night Delivery	Rs 48.00	Net Banking	Placed	2026-04-15 09:11
1	Sarah Williams	Fast Delivery	Rs 76.00	UPI	Placed	2026-04-14 17:42

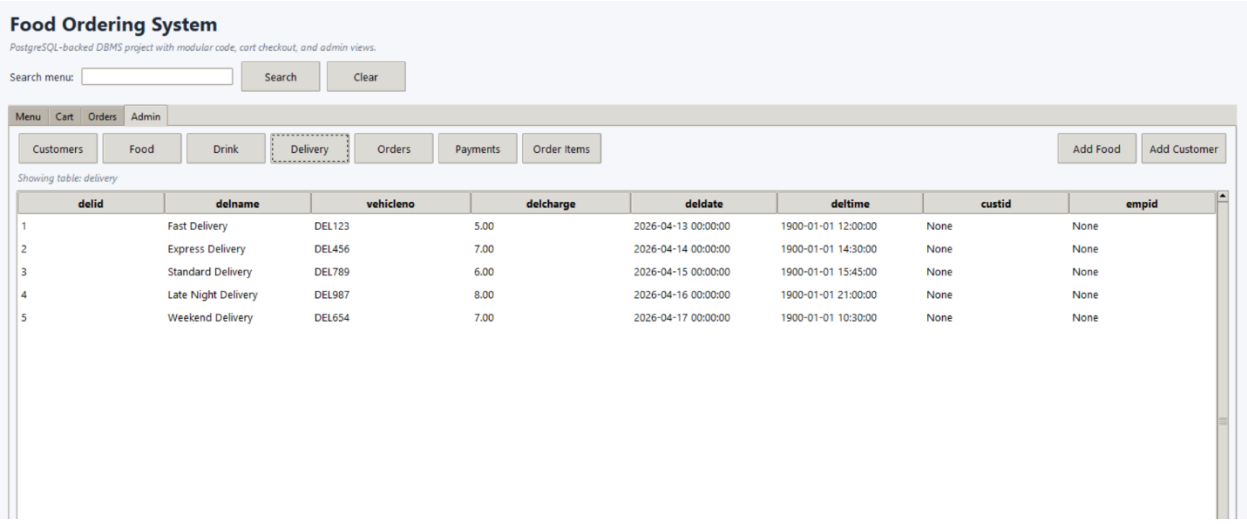
6. Admin Panel

- Admins can view all database tables (customers, food, drinks, orders, order_items, payments, deliveries) in sortable Treeview tables.
- Uses a whitelist-based get_table_rows() function to prevent SQL injection.



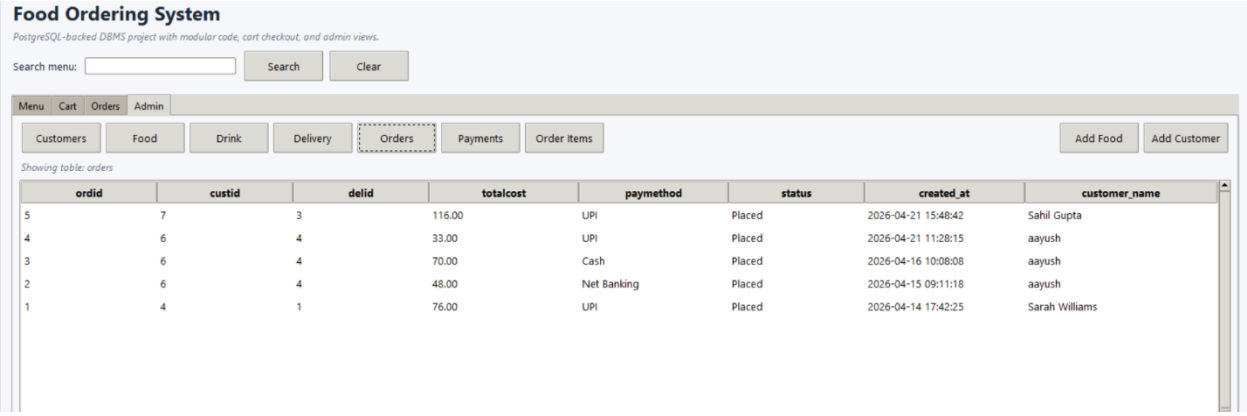
foodid	foodname	price	quantity	foodavail	cuisineid	ingid	chefid
1	Margherita Pizza	12.00	9	Available	1	1	1
2	Kung Pao Chicken	15.00	15	Available	2	2	2
3	Taco	10.00	26	Available	3	3	3
4	Chicken Biryani	14.00	22	Available	4	4	4
5	Sushi Rolls	18.00	16	Available	5	5	5

Admin food menu



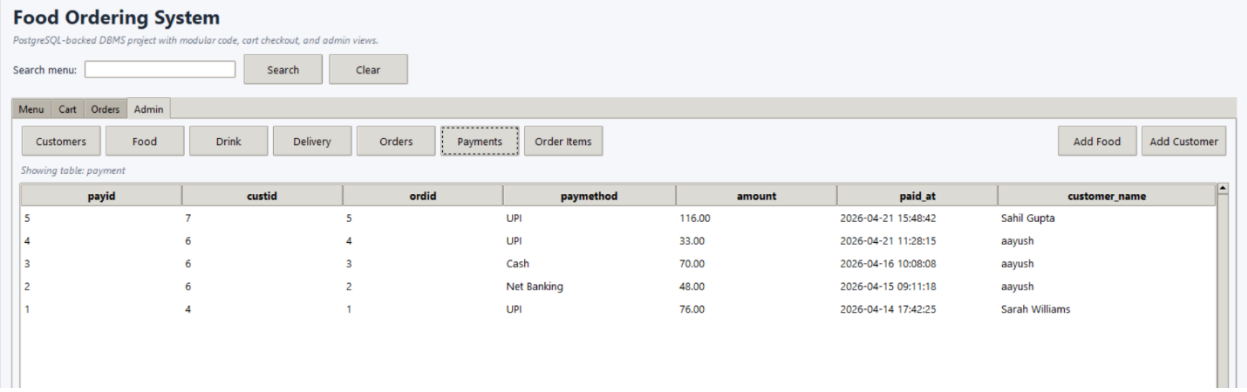
delid	delname	vehicleno	delcharge	deldate	deltime	custid	empid
1	Fast Delivery	DEL123	5.00	2026-04-13 00:00:00	1900-01-01 12:00:00	None	None
2	Express Delivery	DEL456	7.00	2026-04-14 00:00:00	1900-01-01 14:30:00	None	None
3	Standard Delivery	DEL789	6.00	2026-04-15 00:00:00	1900-01-01 15:45:00	None	None
4	Late Night Delivery	DEL987	8.00	2026-04-16 00:00:00	1900-01-01 21:00:00	None	None
5	Weekend Delivery	DEL654	7.00	2026-04-17 00:00:00	1900-01-01 10:30:00	None	None

Admin (delivery)



ordid	custid	delid	totalcost	paymethod	status	created_at	customer_name
5	7	3	116.00	UPI	Placed	2026-04-21 15:48:42	Sahil Gupta
4	6	4	33.00	UPI	Placed	2026-04-21 11:28:15	aayush
3	6	4	70.00	Cash	Placed	2026-04-16 10:08:08	aayush
2	6	4	48.00	Net Banking	Placed	2026-04-15 09:11:18	aayush
1	4	1	76.00	UPI	Placed	2026-04-14 17:42:25	Sarah Williams

Admin (order_details)



payid	custid	ordid	paymethod	amount	paid_at	customer_name
5	7	5	UPI	116.00	2026-04-21 15:48:42	Sahil Gupta
4	6	4	UPI	33.00	2026-04-21 11:28:15	aayush
3	6	3	Cash	70.00	2026-04-16 10:08:08	aayush
2	6	2	Net Banking	48.00	2026-04-15 09:11:18	aayush
1	4	1	UPI	76.00	2026-04-14 17:42:25	Sarah Williams

Admin(payment details)

Performance and Observations

Performance

- System Efficiency: The Food Ordering System performs efficiently. PostgreSQL retrieves and updates data quickly with minimal latency thanks to named indexes on frequently queried columns.
- Response Time: Average response time for operations (menu loading, order placement) is under 2 seconds for normal loads.
- Data Handling: The system manages user details, restaurant menus, and order information through normalized PostgreSQL tables, reducing redundancy and improving consistency.
- Scalability: PostgreSQL's client-server architecture supports multi-user access and can accommodate more restaurants, menu items, and users without major architectural changes.
- Transaction Safety: The checkout is fully ACID-compliant — stock decrement, order creation, and payment recording succeed or fail atomically.

Observations

- Users can easily browse and order food using an intuitive tab-based interface.
- The search bar enables fast menu filtering via PostgreSQL ILIKE queries.
- The database tables work in coordination with proper FK relationships and cascade rules.
- Order status updates (Placed → Preparing → Delivered) are supported via the status column.
- The application demonstrates accurate total calculation (price × quantity per item) during checkout.

10. Conclusion and Future Scope

Conclusion

The Food Ordering Management System successfully provides a convenient, fast, and reliable platform for customers to browse restaurant menus, place food orders, and make secure payments. It has been enhanced from a basic SQLite prototype to a production-grade PostgreSQL-backed application with modular code, transaction-safe checkout, normalized schema design, and live quantity tracking.

The migration from SQLite to PostgreSQL brings significant improvements: full ACID transactions, row-level locking for concurrency safety, named indexes for query performance, and a normalized order_items table that eliminates previous data redundancy.

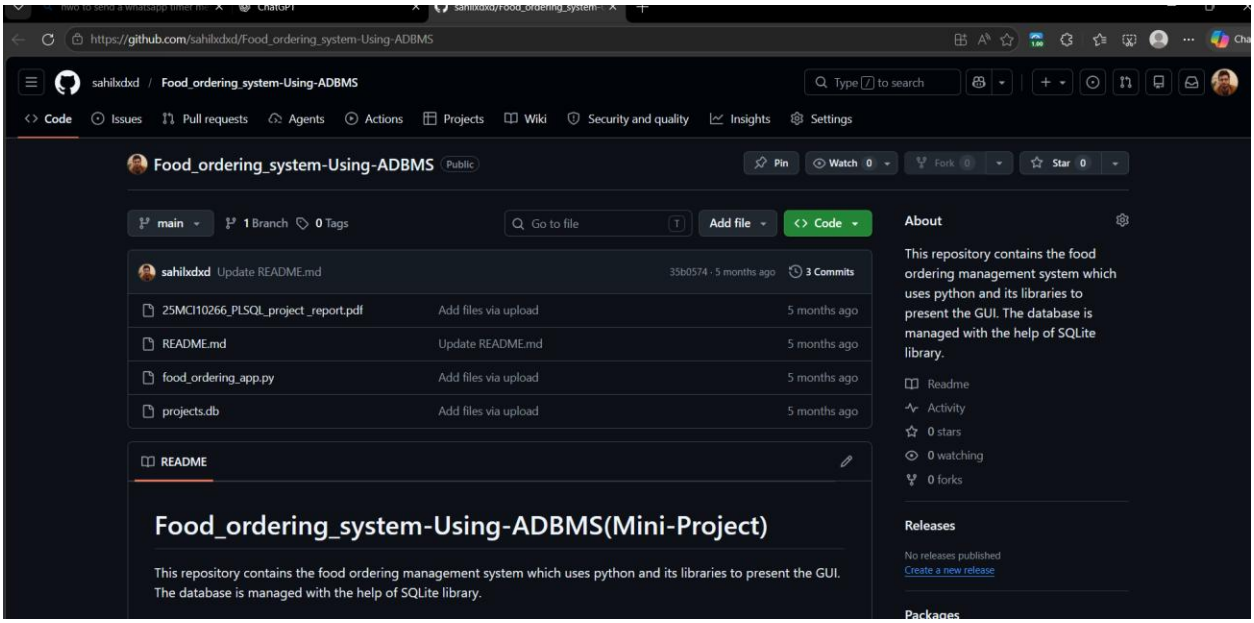
The application's intuitive GUI and well-structured PostgreSQL database ensure smooth operation, efficient data management, and accurate order processing. Overall, the system achieves its primary objectives of improving customer experience, minimizing manual errors, and optimizing restaurant operations through digital automation.

Future Scope

- AI-Based Recommendation System: Integrate artificial intelligence to recommend dishes based on user preferences, order history, and reviews.
- Real-Time GPS Tracking: Add live delivery tracking using GPS to show the delivery partner's location and estimated arrival time.
- Push Notifications: Implement real-time notifications for order confirmation, preparation, and delivery status updates.
- Multi-Restaurant Integration: Expand the system to handle multiple restaurants, each with independent dashboards, menus, and analytics.
- Wallet & Loyalty Points: Introduce a digital wallet or reward system where users earn loyalty points for every order.
- Cloud Deployment: Host the PostgreSQL database and Python application on a cloud platform (AWS RDS + EC2, or Heroku) for scalability and high availability.
- AI Chatbot Support: Provide an AI-powered chatbot to assist users with order placement, complaints, or menu suggestions.
- Machine Learning Analytics: Use ML algorithms with pandas DataFrames to analyze user data for predicting popular items, peak hours, and restaurant performance.
- Web / Mobile Interface: Convert the Tkinter desktop app to a Flask/FastAPI web backend with a React or Flutter mobile frontend for broader accessibility.

GitHub Link

The complete source code for this project is available on GitHub:
https://github.com/sahilxdxd/Food_ordering_system-Using-ADBMS
(Press CTRL + Left Mouse Key to open the link)



12. Bibliography

Books & Study Material

- "Python Programming for Beginners" by John Zelle, Franklin, Beedle & Associates.
- "Learning Python" by Mark Lutz, O'Reilly Media.
- "Database System Concepts" by Abraham Silberschatz, Henry F. Korth, and S. Sudarshan.
- "Fundamentals of Database Systems" by Ramez Elmasri and Shamkant B. Navathe.
- "Modern Database Management" by Jeffrey A. Hoffer, V. Ramesh, Heikki Topi.
- "PostgreSQL: Up and Running" by Regina Obe and Leo Hsu, O'Reilly Media.

Online Resources

- https://docs.python.org/3/ — Official Python Documentation.
- https://www.postgresql.org/docs/ — Official PostgreSQL Documentation.
- https://tkdocs.com/ — Tkinter GUI Programming Tutorials.
- https://www.psycopg.org/docs/ — psycopg2 PostgreSQL adapter documentation.
- https://www.geeksforgeeks.org/python-tkinter-tutorial/ — Tkinter and Python Database Integration.

Research Papers & References

- S. Kumar, "Online Food Ordering System Using Python," International Journal of Emerging Trends in Engineering and Development, Vol. 8, Issue 2, 2022.
- A. Sharma & R. Verma, "Automation in Restaurant Management Using Python and PostgreSQL," International Journal of Computer Applications, 2023.
- P. Singh, "Migration from SQLite to PostgreSQL in Python Applications," International Journal of Advanced Computer Science, 2024.